BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Assembler.

Q2. Target/object code.

Q3. A system that executes/analyses source or intermediate operations at runtime.

Q4. ¬A∧¬B; with A=0,B=1 gives 1∧0=0.

You Do pages · Independent questions

Q1. Tokens [2], grammar/parse [2], type/name and contextual rules [2].

Q2. Generate target instructions [2]; resolve external references and combine object code/libraries [2]; place executable in memory and start [2].

Q3. Lexing tokens valid [1]; parsing structure valid [1]; semantic name/symbol resolution can reject undefined identifier [2]; external symbol might instead fail during linking if declaration exists but definition missing [2].

Exit Ticket · Model answer

False. That is a semantic/name-resolution concern.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

RHS COMPUTING / OCR H446 / mixed LESSON 274 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

LESSON 274 · Compiler stages, linkers, loaders and libraries

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. Which program translates assembly mnemonics?

Q2. What does a compiler typically produce?

Q3. What is a runtime interpreter?

Q4. Simplify ¬(A∨B) with De Morgan's law, then evaluate A=0,B=1.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

lexer — Recognises source tokens.

parser — Checks grammatical structure.

semantic analysis — Checks program meaning and types.

linker — Resolves symbols across modules.

loader — Places program in memory.

New learning

Understand — Lexical analysis identifies tokens, while syntax analysis checks grammar and generates a parse structure.

Apply — Semantic analysis checks rules beyond grammar such as type compatibility or declared identifiers; optimisations can transform intermediate code.

Evaluate — A linker combines object files and resolves symbols/libraries; a loader places the executable into memory for execution.

Worked example

Analyse `total = count + 1;` into tokens and syntax structure.

1. Predict first — The lexer recognises tokens

2. Model answer — parser checks that they form a valid assignment statement.

We Do · Practise with guidance

The program calls PrintReport() from another object file. Which stage resolves the reference?

```
Compiler emits external symbol reference → linker resolves PrintReport
```

Predict first

The linker finds its definition in the module/library and fixes up the reference before execution.

Model answer

Explain the reasoning and justify the output.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Distinguish lexical, syntax and semantic analysis. [6]

Question 2

Describe the roles of code generation, linking and loading. [6]

Question 3

A source file has valid syntax but calls undefined MethodX(). Explain which analyses or stages may detect failure, and why. [6]

Exit Ticket · Answer before checking

A parser guarantees that every variable in a syntactically valid program was declared.